

# Hashing วิธีการนำข้อมูลใส่ใน Array

hash table ข้อมูล Array ที่ใส่เข้าไป

hash function เป็นการออกแบบคีย์ มีคุณสมบัติ 3 ประการ

1. ง่ายต่อการคำนวณ
2. ตั้งมันเข้าได้ว่าข้อมูลที่แตกต่างกัน 2 ตัวต้องได้ตำแหน่งคนละตำแหน่ง ไม่ควมวเป็นจริงเป็นไปได้
3. ช่วงกระจายข้อมูลได้อย่างทั่วถึง

## ข้อสังเกต

1. 0 อยู่บน
2. ข้อมูลที่อยู่ใน Array ไม่จำเป็นต้องเป็นตัวเลข

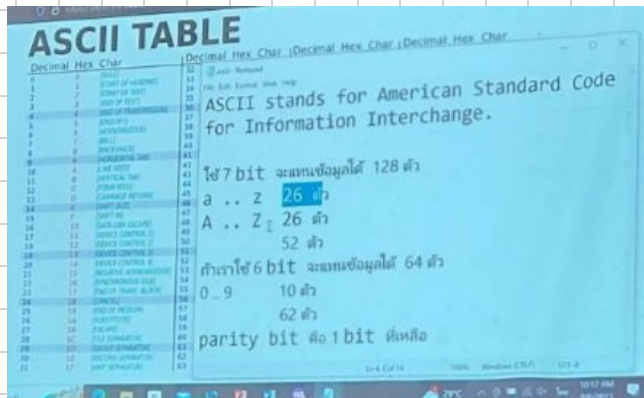
หลักการ wrap around ใน Hashing ก็คือ เมื่อคำนวณ ถึงตำแหน่งท้ายของ Array แล้วให้วนกลับมา ตำแหน่งที่ 0 ใหม่ ก็คือ  $\text{mod tablesize}$  อีกรอบหนึ่งนั่นเอง

## main program

```
int hash(const string & key, int tablesize)
int main
{
    String a[7];
    int x;
    x = hash("ANN", 7)
    a[x] = ("ANN")
    return 0;
}
```

mod

if ตัวนี้มันจะบอกว่า คำตอบคือตัวหน้า  
เช่น  $9 \text{ mod } 10 = 9$



## โปรแกรมที่สร้าง for

ในการเก็บข้อมูล for มี 3 ส่วน แต่ละส่วนนั้นด้วย ;

- ส่วนที่ 1 ส่วนกำหนดค่า
  - ส่วนที่ 2 ส่วนของเงื่อนไข
  - ส่วนที่ 3 ส่วนที่จะเปลี่ยนแปลงในเมื่อจบรอบ
- ของ for loop
- \* รอบแรก ส่วนที่ 3 ไม่ทำ
- ส่วนที่ 1 จากเมื่อรอบแรก
- ส่วนที่ 1 , ส่วนที่ 2 ; ส่วนที่ 3

## Hash function

- If keys are strings ข้อความ คีย์
- Add ASCII values of the ch
- If TableSize is large and nur
- TableSize = 10000 & n
- 127\*8=1016 < 10000

ส่วนที่สาม

```
int hash( const string & key
{
    int hashVal = 0;
```

hashVal = 0

for loop รอบแรก  $i = 0 ; 0 < 3$

hashVal = hashVal + key [0];

= 0 + 'A';

= 0 + 65;

เลขฐานสิบค่าที่ใช้ในวิธีตาราง: จำนวน Dec

hashVal = 65

for loop รอบที่ 2  $i = 1 ; 1 < 3$  ทำส่วนที่ 3 ก่อน

hashVal = hashVal + key [1];

= 65 + 'N';

= 65 + 78;

hashVal = 143

for loop รอบที่ 3  $i = 2 ; 2 < 3$

hashVal = hashVal + key [2];

= 143 + 'N';

= 143 + 78;

hashVal = 221

for loop รอบที่ 4  $i = 3 ; 3 < 3$  เท็จ

ออกจาก for loop ไป return  $221 \% 7 = 4$

ที่ซ่อนใน for loop เท่ากับ 3 รอบ

ออกจาก for loop ตัวแปร  $i$  มีค่าเท่ากับ 3 ไม่ใช่ 4

local variable ตัวแปรเฉพาะที่  
ถ้า  $i$  อยู่นอก loop มีค่า

ออกจาก for loop hashVal มีค่าเท่ากับ 221

|     |   |     |
|-----|---|-----|
| 111 | 0 |     |
|     | 1 |     |
|     | 2 |     |
|     | 3 |     |
|     | 4 | ANN |
|     | 5 |     |
|     | 6 |     |

## Collision กรณีชนกัน

Open Addressing ไปที่ช่องอื่นแทน

$$h_i(x) = \text{hash}(x) + F(i) \mod \text{TableSize}$$

$$F(0) = 0$$

### 1. Linear Probing วิธีเส้น ไม่ใส่เลขยกกำลัง

ปัญหาของ Linear Probing

1. ใส่รอบแต่ลบยาก

2. Primary Clustering การเกาะกลุ่มกันแรก ขยายถึง เลื่อนมีที่ซ่อนกันเกิดขึ้น ทางออกต่างในล้น  
หากพื้นที่จะเกิดการเกาะกลุ่มกัน

## 2. Quadratic Probing

$$F(i) = i^2$$

89, 18, 49, 58( ), 69( )

$$h_i(x) = (\text{hash}(x) + F(i)) \% 10; F(0) = 0$$

$$h_0(49) = ((49 \% 10) + F(0)) \% 10;$$

$$h_0(49) = (9 + 0) \% 10 = 9 \% 10 = 9$$

89, 18, 49, 58( ), 69( )

$$h_i(x) = (\text{hash}(x) + F(i)) \% 10; F(0) = 0$$

$$h_0(18) = ((18 \% 10) + F(0)) \% 10;$$

$$h_0(18) = (8 + 0) \% 10 = 8 \% 10 = 8$$

89, 18, 49, 58( ), 69( )

$$h_i(x) = (\text{hash}(x) + F(i)) \% 10; F(0) = 0$$

$$h_0(49) = ((49 \% 10) + F(0)) \% 10;$$

$$h_0(49) = (9 + 0) \% 10 = 9 \% 10 = 9 \quad F(i) = i^2$$

$$h_1(49) = (9 + 1) \% 10 = 10 \% 10 = 0 \quad F(1) = 1^2$$

89, 18, 49, 58(2), 69( )

$$h_i(x) = (\text{hash}(x) + F(i)) \% 10; F(0) = 0$$

$$h_0(58) = ((58 \% 10) + F(0)) \% 10;$$

$$h_0(58) = (8 + 0) \% 10 = 8 \% 10 = 8 \quad F(i) = i^2$$

$$h_1(58) = (8 + 1) \% 10 = 9 \% 10 = 9 \quad F(1) = 1^2$$

$$h_2(58) = (8 + 4) \% 10 = 12 \% 10 = 2 \quad F(2) = 2^2$$

## 3. Double Hashing

$$F(i) = i * \text{hash}_2(x)$$

$$\text{hash}_2(x) = R - (x \bmod R)$$

R is a จำนวนเฉพาะและน้อยกว่า TableSize

ถ้า TableSize เป็น 10, R จะเท่ากับ 7

ถ้า 11 R จะเท่ากับ 7

tableSize

89, 18, 49, 58, 69

$$h_i(x) = (\text{hash}_1(x) + F(i)) \% 10; F(0) = 0$$

$$h_0(49) = ((49 \% 10) + F(0)) \% 10;$$

$$h_0(49) = (9 + 0) \% 10 = 9 \% 10 = 9$$

$$F(1) = 1 * \text{hash}_2(x) = 7 - (49 \bmod 7)$$

$$F(1) = 1 * 7 = 7 - 0 = 7$$

$$h_1(49) = (9 + 7) \% 10 = 16 \% 10 = 6 \quad \text{ข้อที่ 6}$$

49 มีที่เก็บแล้ว แต่มีที่ว่าง 1 ครั้ง มีที่ว่างที่ 6 ครั้งนี้ 1 ไปหาที่ว่างที่ 6

89, 18, 49, 58, 69

$$h_i(x) = (\text{hash}_1(x) + F(i)) \% 10; F(0) = 0$$

$$h_0(58) = ((58 \% 10) + F(0)) \% 10;$$

$$h_0(58) = (8 + 0) \% 10 = 8 \% 10 = 8$$

$$F(1) = 1 * \text{hash}_2(x) = 7 - (58 \bmod 7)$$

$$F(1) = 1 * 5 = 7 - 2 = 5$$

$$h_1(58) = (8 + 5) \% 10 = 13 \% 10 = 3 \quad \text{ข้อที่ 3}$$

58 มีที่เก็บแล้ว แต่มีที่ว่าง 1 ครั้ง มีที่ว่างที่ 3 ครั้งนี้ 1 ไปหาที่ว่างที่ 3

$$\text{hash}_1(x) = x \% \text{tableSize}$$

ถ้า R เป็นจำนวนเฉพาะ < TableSize  
ถ้า R เป็นจำนวนเฉพาะ > TableSize

89, 18, 49, 58, (69), 60, 23

$$h_i(x) = (\text{hash}_i(x) + F(i)) \% 10; F(0) = 0$$

$$h_0(69) = ((69 \% 10) + F(0)) \% 10; F(i) = i * \text{hash}_2(x) = i * (R - (x \% R))$$

$$h_0(69) = (9 + 0) \% 10 = 9 \% 10 = 9$$

$$F(1) = 1 * \text{hash}_2(x) = 7 - (69 \% 7)$$

$$F(1) = 1 * 1 = 7 - 6 = 1$$

$$h_1(69) = (9 + 1) \% 10 = 10 \% 10 = 0 \text{ ข้อที่ 0}$$

69 มีการชนกันและเกิดการชนกัน 1 ครั้ง การแก้การชนกันครั้งที่ 1 ไปตรวจหาช่องว่างใหม่ รวบรวม  $F(1) = 1$  ไปเก็บข้อที่ 0

89, 18, 49, 58, 69, (60), 23

$$h_i(x) = (\text{hash}_i(x) + F(i)) \% 10; F(0) = 0$$

$$h_0( ) = (( ) \% 10) + F(0)) \% 10; F(i) = i * \text{hash}_2(x) = i * (R - (x \% R))$$

$$h_0( ) = ( + 0) \% 10 = \% 10 =$$

$$F(1) = 1 * \text{hash}_2(x) = 7 - (60 \% 7)$$

$$F(1) = 1 * 5 = 7 - 2 = 5$$

$$h_1(60) = (0 + 5) \% 10 = 5 \% 10 = 5 \text{ ข้อที่ 5}$$

60 มีการชนกันและเกิดการชนกัน 1 ครั้ง การแก้การชนกันครั้งที่ 1 ไปตรวจหาช่องว่างใหม่ รวบรวม  $F(1) = 5$  ไปเก็บข้อที่ 5

89, 18, 49, 58, 69, (60), 23

$$h_i(x) = (\text{hash}_i(x) + F(i)) \% 10; F(0) = 0$$

$$h_0(60) = ((60 \% 10) + F(0)) \% 10; F(i) = i * \text{hash}_2(x) = i * (R - (x \% R))$$

$$h_0(60) = (0 + 0) \% 10 = 0 \% 10 = 0$$

$$F(1) = 1 * \text{hash}_2(x) = 7 - (60 \% 7)$$

$$F(1) = 1 * 3 = 7 - 4 = 3$$

$$h_1(60) = (0 + 3) \% 10 = 3 \% 10 = 3$$

$$h_2(60) = (0 + 6) \% 10 = 6 \% 10 = 6$$

$$h_3(60) = (0 + 9) \% 10 = 9 \% 10 = 9$$

$$h_4(60) = (0 + 12) \% 10 = 2 \% 10 = 2$$

$$F(2) = 2 * 3 = 7 - (60 \% 7)$$

$$F(2) = 2 * 3 = 7 - 4 = 3$$

$$= 6 = 7 - (60 \% 7)$$

$$F(3) = 3 * 3 = 7 - 4 = 3$$

$$= 9 = 7 - 4 = 3$$

$$= 12 = 7 - 4 = 3$$

$$F(4) = 4 * 3 = 7 - 4 = 3$$

$$= 12 = 7 - 4 = 3$$

60 มีการชนกันและเกิดการชนกัน 1 ครั้ง การแก้การชนกันครั้งที่ 4 ไปตรวจหาช่องว่างใหม่ รวบรวม  $F(1) = 4$  ไปเก็บข้อที่ 2



89, 18, 49, 58, 69, (60), 23

$$h_i(x) = (\text{hash}_i(x) + F(i)) \% 10; \quad F(0) = 0$$

$$h_0(60) = ((60 \% 10) + F(0)) \% 10; \quad F(i) = i * \text{hash}_i(x) = i * (R - (x \% R))$$

$$h_0(60) = (0 + 0) \% 10 = 0 \% 10 = 0$$

$$F(1) = 1 * \text{hash}_1(x) = 1 - (60 \% 7)$$

$$F(1) = 1 * 3 = 1 - 4 = 3$$

$$h_1(60) = (0 + 3) \% 10 = 3 \% 10 = 3$$

$$F(2) = 2 * 3 = 2 - (60 \% 7)$$

$$h_2(60) = (0 + 6) \% 10 = 6 \% 10 = 6$$

$$F(2) = 2 * 3 = 2 - 4 = 3$$

$$h_3(60) = (0 + 9) \% 10 = 9 \% 10 = 9$$

$$F(3) = 3 * 3 = 3 - (60 \% 7)$$

$$h_4(60) = (0 + 12) \% 10 = 2 \% 10 = 2$$

$$F(3) = 3 * 3 = 3 - 4 = 3$$

$$F(4) = 4 * 3 = 4 - 4 = 3$$

60 มีการชนกันแล้วแต่ไม่มีการชนกันจริงๆ เพราะมีการกระจายตัวที่ดี 4 / 10 แต่วางของไว้ที่ 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100

## Type declaration for open addressing hash table

```
template <class HashedObj>
class HashTable
{
public:
    explicit HashTable(const HashedObj & notFound, int size = 101);
    HashTable(const HashTable & rhs)
        : ITEM_NOT_FOUND(rhs.ITEM_NOT_FOUND), array( rhs.array ),
          currentSize( rhs.currentSize ) {}

    const HashedObj & find( const HashedObj & x ) const;

    void makeEmpty();
    void insert( const HashedObj & x );
    void remove( const HashedObj & x );

    const HashTable & operator=( const HashTable & rhs );

    enum EntryType { ACTIVE, EMPTY, DELETED };
};
```

```
private:
    struct HashEntry
    {
        HashedObj element;
        EntryType info;
    };
    HashEntry( const HashedObj & e = HashedObj(),
               EntryType i = EMPTY ) : element( e ), info(i) {}

    vector<HashEntry> array;
    int currentSize;
    const HashedObj ITEM_NOT_FOUND;

    bool isActive( int currentPos ) const;
    int findPos( const HashedObj & x ) const;
    void rehash();
};
```

```

/* Construct the hash table.
template <class HashedObj>
HashTable<HashedObj>::
HashTable( const HashedObj & notFound, int size )
:ITEM_NOT_FOUND( notFound ), array( nextPrime( size ) )
{
    makeEmpty( );
}

/* Make the hash table logically empty.
template <class HashedObj>
void HashTable<HashedObj>::makeEmpty( )
{
    currentSize = 0;
    for( int i = 0; i < array.size( ); i++ )
        array[ i ].info = EMPTY;
}

```

```

template <class HashedObj>
const HashedObj & HashTable<HashedObj>::
find( const HashedObj & x = ) const {
    int currentPos = findPos( x = );
    if( isActive( currentPos ) )
        return array[ currentPos ].element;
    else return ITEM_NOT_FOUND;
}

/* Method that performs quadratic probing resolution.  $f(i) = i^2$ 
template <class HashedObj>
int HashTable<HashedObj>::findPos( const HashedObj & x ) const
{
    int collisionNum = 0; //ก็คือ ตัวแปร i ในสมการ นั่นเอง
    int currentPos = hash( x, array.size( ) );
    currentPos = 5;
    while ( array[ currentPos ].info != EMPTY &&
            array[ currentPos ].element != x ) {
        currentPos += 2 * ++collisionNum - 1;

        if( currentPos >= array.size( ) )
            currentPos -= array.size( );
    }
}

```

```

template <class HashedObj>
bool HashTable<HashedObj>::isActive( int currentPos ) const
{
    return array[ currentPos ].info == ACTIVE;
}

/* Remove item x from the hash table.
template <class HashedObj>
void HashTable<HashedObj>::remove( const HashedObj & x )
{
    int currentPos = findPos( x );
    if( isActive( currentPos ) )
        array[ currentPos ].info = DELETED;
    currentSize--;
}

/* Insert routine with quadratic probing
template <class HashedObj>
void HashTable<HashedObj>::insert( const HashedObj & x ) {
    int currentPos = findPos( x );
    if( isActive( currentPos ) ) เป็นเท็จ ไม่ทำ return;
    array[ currentPos ] = HashEntry( x, ACTIVE );
    if( ++currentSize > array.size( ) / 2 )
        rehash( );
}

```

```

/* Deep copy.
template <class HashedObj>
const HashTable<HashedObj> & HashTable<HashedObj>::
operator=( const HashTable<HashedObj> & rhs )
{
    if( this != &rhs )
    {
        array = rhs.array;
        currentSize = rhs.currentSize;
    }
    return *this;
}

```

# Rehashing

วิธีแก้ ถ้า table ใหม่ที่หาได้น้อยกว่าเดิมสองเท่า

- จำนวนช่องใน hash table มี 7 ช่อง ใช้ไปแล้ว 4 ช่อง
- จำนวนช่องใน hash table มี 100 คิดเป็น  $4 * 100 / 7 = 57.14 \%$

ถ้า 23 ช่อง 2

- จำนวนช่องใน hash table มี 7 ช่อง ใช้ไปแล้ว 5 ช่อง
- จำนวนช่องใน hash table มี 100 คิดเป็น  $5 * 100 / 7 = 71.42 \%$

ถ้ามากกว่า 70% ก็ re

Table เดิม มี 7 ช่อง

Table ใหม่ ที่เพิ่มเข้ามาเพิ่มเป็น 2 เท่า ก็ควรเป็น 14 ช่อง

แต่ถ้ามี 13, 14, 15, 16, 17

เลือก Table Size เป็นเลขจำนวนเฉพาะ: น้อยที่สุด แต่มากกว่า หรือ เท่ากับ 2 เท่า ของ

Hash Table เดิม

อ่านข้อมูลจาก Hash Table เดิม แล้วย้ายเป็น 6, 15, 23, 24, 13

$$H_{\text{new}}(6) = 6 \text{ mod } 17$$

ค่าที่ได้จากค่าเดิมของ Input ข้อมูลนั้น: ครบ หมายถึง ไม่ใช้

13, 15, 27, 6, 23 คือ

|   |    |
|---|----|
| 0 | 6  |
| 1 | 15 |
| 2 |    |
| 3 | 24 |
| 4 |    |
| 5 |    |
| 6 | 13 |

|   |    |
|---|----|
| 0 | 6  |
| 1 | 15 |
| 2 | 23 |
| 3 | 24 |
| 4 |    |
| 5 |    |
| 6 | 13 |

## Rehashing Example

- If 23 is inserted, the table is over **70** percent full. มากกว่า 70% ก็ re

|   |    |
|---|----|
| 0 | 6  |
| 1 | 15 |
| 2 | 23 |
| 3 | 24 |
| 4 |    |
| 5 |    |
| 6 | 13 |



A new table is created

17 is the first prime twice as large as the old one; so

$$H_{\text{new}}(X) = X \text{ mod } 17$$

|    |    |
|----|----|
| 0  |    |
| 1  |    |
| 2  |    |
| 3  |    |
| 4  |    |
| 5  |    |
| 6  | 6  |
| 7  | 23 |
| 8  | 24 |
| 9  |    |
| 10 |    |
| 11 |    |
| 12 |    |
| 13 | 13 |
| 14 |    |
| 15 | 15 |
| 16 |    |

ขั้นตอนที่ 1 ระบบปฏิบัติการคอมพิวเตอร์ จะทำการสร้างอาร์เรย์ของบิตที่ a ขึ้นในหน่วยความจำ ตามโครงสร้าง คลาส HashTable ชนิดข้อมูล int จะถูกส่งเข้าไปที่ class template ชื่อ HashedObj ของไหนเป็นค่า HashedObj ในโครงสร้าง คลาส HashTable ก็จะถูกเปลี่ยนเป็นค่า int สำหรับตัวแปรของบิตที่ a มีการส่งค่าหาบิตที่ a ไป 2 ค่า คือ 0-999 กับ 10

ขั้นตอนที่ 2 ระบบปฏิบัติการจะใส่ฟังก์ชัน Constructure function ให้กับ HashTable

```
template < class HashedObj >
```

```
HashTable < HashedObj > :: HashTable ( const HashedObj & notFound, int size)
```

```
: ITEM NOT FOUND (notFound), array (nextPrime (size))
```

```
{
```

```
makeEmpty ( ) ;
```

```
}
```

## Priority Queues (HEAPS)

### complete binary tree

binary tree ที่เต็มหมดทุกชั้น

ยกเว้นชั้นล่างสุดที่ไม่เต็มก็ได้

เวลาใส่ข้อมูล จะได้ที่ขึ้นลงสุดจากซ้ายไปขวา

เวลาลบข้อมูล จะลบที่ชั้นล่างสุด จากขวาไปซ้าย

ช่องที่ 0 ไม่ใส่ข้อมูล , บนลงล่างซ้ายไปขวา , ตำแหน่งเป็นเลขที่ใช้ตัดเศษทิ้ง

parent หรือ node child

Complete Binary Tree ที่มีค่าของสูง  $H = 15$

จำนวนโหนดของบิต  $= 2^H = 32 \times 32 \times 32 = 32768$  โหนด

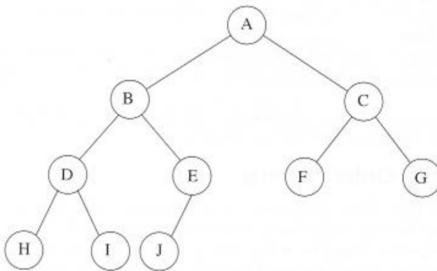
จำนวนโหนดของบิต  $2^H - 1$

ถ้าติดในรูปเลขยกกำลัง จะได้อะไรแน่นอน

## Structure Property of Heaps

Array implementation of complete binary tree

บนลงล่างซ้ายไปขวา



ช่องที่ 0 ไม่ใส่ข้อมูล

■ For an element in position  $i$

■  $2i$  is the left child

■  $2i+1$  is the right child

■  $i/2$  is the parent

ตัวอย่างที่  $7/2 = 3$

ตำแหน่งเป็นเลขคู่จึงตัดเศษทิ้ง /

array implementation requires an estimate on **max-size**

|   | A | B | C | D | E | F | G | H | I | J  |    |    |    |  |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|--|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |  |

int main

{

BinaryHeap<int> a(12); ← 12 จำนวน 12 ตัว  
int x,  
a.insert(14);  
a.deleteMin(x);

111 0

1

2

3

4

5

6

7

8

10

11

12

template <class Comparable>

```
class BinaryHeap {  
public:  
    explicit BinaryHeap( int capacity = 100 );
```

```
    bool isEmpty( ) const;  
    bool isFull( ) const;  
    const Comparable & findMin( ) const;
```

```
    void insert( const Comparable & x );  
    void deleteMin( );  
    void deleteMin( Comparable & minItem );  
    void makeEmpty( );
```

```
private:
```

```
    int currentSize;  
    vector<Comparable> array;  
    void buildHeap( );  
    void percolateDown( int hole );
```

```
/* Construct the binary heap.  
template <class Comparable>  
BinaryHeap<Comparable>::BinaryHeap( int capacity )  
    : array( capacity + 1 ), currentSize( 0 ) { }
```

```
/*Test if the priority queue is logically empty.  
template <class Comparable>  
bool BinaryHeap<Comparable>::isEmpty( ) const {  
    return currentSize == 0;  
}
```

```
/*Test if the priority queue is logically full.  
template <class Comparable>  
bool BinaryHeap<Comparable>::isFull( ) const {  
    return currentSize == array.size( ) - 1; // 12  
                                           10      15      12  
                                           คือ
```

```
/*Make the priority queue logically empty.  
template <class Comparable>  
void BinaryHeap<Comparable>::makeEmpty( ) {  
    currentSize = 0;
```

ขั้นตอนที่ 1 ระบบปฏิบัติการคอมพิวเตอร์ จะทำการสร้างตัวแปรของเจกต์ a ขึ้นในหน่วยความจำ  
 ตามโครงสร้างข้อมูล BinaryHeap  
 ชนิดข้อมูล int จะถูกส่งเข้าไปที่ class template ชื่อ Comparable ตรงไหนเป็นค่า Comparable  
 โครงสร้าง คลาส BinaryHeap ก็จะถูกเปลี่ยนเป็นค่า int สำหรับตัวแปรของเจกต์ a

a

currentSize =

array =

ขั้นตอนที่ 2 ระบบปฏิบัติการจะไปทวน Constructure function ที่กั้น BinaryHeap

template < class Comparable >

BinaryHeap < Comparable > :: BinaryHeap (int capacity) : array( capacity + 1 ,  
 currentSize(0) ) { }

ตัวแปรพารามิเตอร์ชื่อ capacity มีค่าเท่ากับ 12

ระบบปฏิบัติการจะไปจองพื้นที่ในหน่วยความจำติดกัน เท่ากับ capacity + 1 เท่ากับ 13 แต่จะช่องมีขนาด  
 เท่ากับชนิดข้อมูล int

เก็บ address ของหน่วยความจำตำแหน่งที่ 0 ไว้ใน data member ชื่อ array  
 เก็บค่า 0 ไว้ใน data member ชื่อ currentSize

การ insert

ขั้นตอน

1. สร้าง hole ที่ตำแหน่งถัดไปใน array
2. จนกว่าจะเป็นรอบ วนได้ x 2 ไปใน hole  
 - การเลื่อน hole ขึ้นไป
3. สอดคอดังได้ x เข้าไปกับ heap parent  
 9 ค่าข้อมูลขึ้นค่าสูงสุดจากข้างไปขวา ข้างขวา  
 ไปลงให้เป็นตัวรองของค่าแรกสุด

```
/* Insert item x into the priority queue
template <class Comparable>
void BinaryHeap<Comparable>::insert( const Comparable&x )
{
    if ( isFull() )
        throw Overflow(); ยังไม่ทำ เพราะ if isfull เที่ย

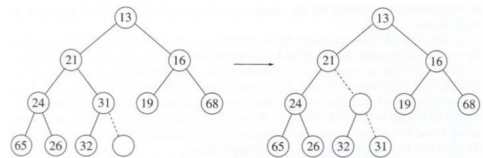
    // Percolate up
    int hole = ++currentSize; + ขึ้นไป 1 ก่อน hole = hole / 2
    for ( ; hole > 1 && x < array[ hole / 2 ]; hole /= 2 )
        array[ hole ] = array[ hole / 2 ];
    array[ hole ] = x;
}
```



## Example

- Attempt to insert 14
- Creating a hole
- Bubbling the hole up

นี่คือขั้นตอนที่สร้างได้ค่าที่ไปลง

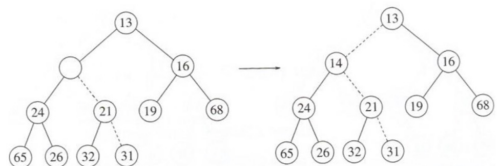


13



## Example

Remaining two steps to insert 14



insert 14 currentSize = 10 → 11

|        |    |       |
|--------|----|-------|
| 111    | 0  | →     |
|        | 1  | 13    |
| ① hole | 2  | 21 14 |
|        | 3  | 16    |
|        | 4  | 24    |
| ① hole | 5  | 31 21 |
|        | 6  | 19    |
|        | 7  | 68    |
|        | 8  | 65    |
|        | 9  | 26    |
| ① hole | 10 | 32    |
|        | 11 | 31    |
|        | 12 |       |

มาทำงานในฟังก์ชัน insert หน้า 19

hole = 11

for loop รอบที่ 1 hole = 11, 11 > 3 **จริง** เปรียบเทียบ 14 < 31 **จริง**  
 for loop รอบที่ 2 hole = 5, 5 > 1 **จริง** เปรียบเทียบ 14 < 21 **จริง**  
 for loop รอบที่ 3 hole = 1, 1 > 1 **จริง** เปรียบเทียบ 14 < 31 **เท็จ**  
 ออกจาก for loop หักแล้ว hole มีค่าเท่ากับค่าใหม่ = 2  
 for loop ทำงานที่รอบ 2 รอบ นับรอบที่เข้าจริง

ณ เวลานี้ ตัวประกอบเจด a มีค่าเท่ากับ 11 = currentSize

ms deleteMin ลบค่าที่น้อยสุด

มาทำงานในฟังก์ชัน percolateDown

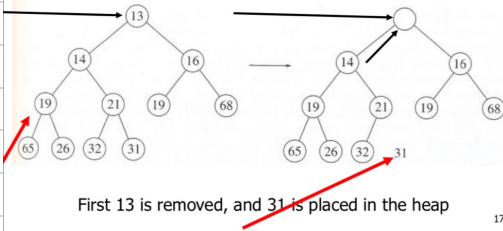
|      |    |       |
|------|----|-------|
| 111  | 0  |       |
|      | 1  | 13    |
|      | 2  | 14 19 |
|      | 3  | 16    |
|      | 4  | 19 26 |
|      | 5  | 21    |
|      | 6  | 19    |
|      | 7  | 68    |
|      | 8  | 65    |
| hole | 9  | 26 31 |
|      | 10 | 32    |
|      | 11 | 31    |
|      | 12 |       |

hole = 1  
 trip = 31  
 for loop รอบที่ 1 hole = 1, 2 < 10 child = 2 เปรียบเทียบ 16 < 14 **เท็จ**  
 for loop รอบที่ 2 hole = 2, 4 < 10, child = 2 \* 2 = 4 เปรียบเทียบ 21 < 19 **เท็จ** child++  
 for loop รอบที่ 3 hole = 4, 4 < 10, child = 8 เปรียบเทียบ 26 < 19 **จริง** child++  
 for loop รอบที่ 4 hole = 9, 14 < 10 **เท็จ** ออกจาก for loop  
 ออกจาก for loop child มีค่าเท่ากับ 9  
 for loop ทำงาน 3 รอบ

ณ เวลานี้ ตัวประกอบเจด a มีค่าเท่ากับ 10 ตัว  
 currentSize = 10 ตัว

## Example

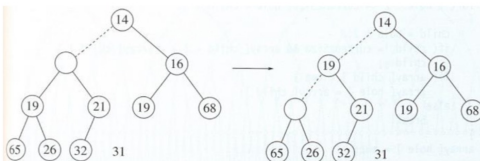
Creation of the hole at the root



17

## Example

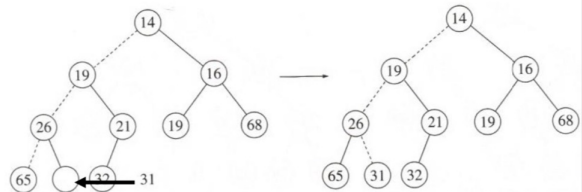
Next two steps in deleteMin



18

## Example

Last two steps in deleteMin



19

```

/* Remove the smallest item from the priority queue
template <class Comparable>
void BinaryHeap<Comparable>::deleteMin( Comparable & minItem ) {
    if( isEmpty() ) throw Underflow();
    minItem = array[ 1 ];
    array[ 1 ] = array[ currentSize-- ];
    percolateDown( 1 );
}

/* Internal method to percolate down in the heap.
template <class Comparable>
void BinaryHeap<Comparable>::percolateDown( int hole ) {
    int child;
    Comparable tmp = array[ hole ];
    for( ; hole * 2 <= currentSize; hole = child ) {
        child = hole * 2;
        if( child != currentSize && array[child + 1] < array[child] ) child++;
        if( array[ child ] < tmp ) array[ hole ] = array[ child ];
        else break;
    }
    array[ hole ] = tmp;
}
    
```



# Sorting

## Preliminaries ข้อควรเบื้องต้น

- ส่งข้อมูลแบบ Array
- เปรียบเทียบขนาดมากกว่า

เวลาพูดถึงการเรียงลำดับทั่วไป เรียงจากน้อยไปมาก

## Insertion Sort

$P$  = Position ในแต่ละรอบมีการเคลื่อนที่  
ก็รอบ

### Insertion Sort

| Original      | 34 | 8  | 64 | 51 | 32 | 21 | P. Moved |
|---------------|----|----|----|----|----|----|----------|
| After $p = 1$ | 8  | 34 | 64 | 51 | 32 | 21 | 1        |
| After $p = 2$ | 8  | 34 | 64 | 51 | 32 | 21 | 0        |
| After $p = 3$ | 8  | 34 | 51 | 64 | 32 | 21 | 1        |
| After $p = 4$ | 8  | 32 | 34 | 51 | 64 | 21 | 3        |
| After $p = 5$ | 8  | 21 | 32 | 34 | 51 | 64 | 4        |

$x = a$

|    |   |    |    |    |    |
|----|---|----|----|----|----|
| 34 | 8 | 64 | 51 | 32 | 21 |
| 0  | 1 | 2  | 3  | 4  | 5  |

For  $\forall p; p = 1; 1 < 6$  จริง

$tmp = 8$

$j =$   
For  $\forall j; j = p = j = 1; 1 > 0 \&\& 8 < a[0] = 34$ ; จริง  
 $a[j+1] = a[j-1] = 0$ ;

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 34  | 8   | 64  | 51  | 32  | 21  |
| [0] | [1] | [2] | [3] | [4] | [5] |

For  $\forall j; j > 0; 0 > 0$  จริง

After  $p = 1$

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 8   | 34  | 64  | 51  | 32  | 21  |
| [0] | [1] | [2] | [3] | [4] | [5] |

For  $\forall p; p = 2; 2 < 6$  จริง

$tmp = 64$

$j =$   
For  $\forall j; j = 2; 2 > 0 \&\& tmp < a[j-1]$  จริง  
position moved = 0

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 8   | 34  | 64  | 51  | 32  | 21  |
| [0] | [1] | [2] | [3] | [4] | [5] |

```
template <class Comparable>
void insertionSort( vector<Comparable> & a )
{
    /* 1*/     for( int p = 1; p < a.size(); p++ )
    {
        /* 2*/         Comparable tmp = a[ p ];

        int j;
        /* 3*/         for( j = p; j > 0 && tmp < a[ j - 1 ]; j-- )
        /* 4*/             a[ j ] = a[ j - 1 ];
        /* 5*/         a[ j ] = tmp;
    }
}

//Insertionsort routine
```

# in  
using  
void insertionSort( vector< Com... > &a );  
int main()

{ vector<int> x(6);

x[0] = 34;

x[1] = 8;

เราเก็บค่าใน Array ตั้งแต่ตำแหน่งที่ 0

insertionSort(x);

position moved = 0

For  $\forall j \mid p = 3; 3 < 6$  <sup>จริง</sup>

$tmp = 51$

$j$   
For  $\forall j = 5; j > 0 \ \&\& \ tmp < a[j-1];$  <sup>จริง</sup>

$a[j] = a[j-1];$

$a[3] = 64$

position moved = 1

After  $p = 3$

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 8   | 34  | 51  | 64  | 92  | 21  |
| [0] | [1] | [2] | [3] | [4] | [5] |

For  $\forall j \mid p = 4;$

$tmp = 32$  position moved = 3

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 8   | 32  | 34  | 51  | 64  | 21  |
| [0] | [1] | [2] | [3] | [4] | [5] |

For  $\forall j \mid p = 5;$   
 $tmp = 21$  position moved = 4

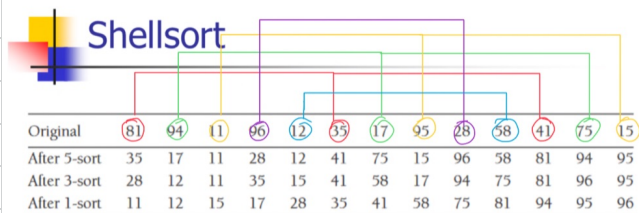
|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 8   | 21  | 32  | 34  | 51  | 64  |
| [0] | [1] | [2] | [3] | [4] | [5] |

จะเขียนค่าในตัวของ tmp จะหามาหรอบที่  $p =$   
ตัวอะไร tmp จะหามาหรอบที่  $p =$

3 case in algorithms เปรียบเทียบ

- Best Case กรณีที่ลิสต์ที่สุ่ม สำหรับ insertionSort ตัวเลข 6 ตัว ข้อมูล Input เป็นข้อมูลที่เรียงแล้วจากน้อยไปมาก
- Worst Case กรณีที่ลิสต์ที่สุ่ม สำหรับ insertionSort ตัวเลข 6 ตัว ข้อมูล Input เป็นข้อมูลที่เรียงแล้วจากมากไปน้อย
- Average Case ข้อมูลทั่วไป

## Shellsort

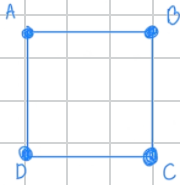


# Graphs

$G =$  คอลเลกชันของจุดกับเส้น

$G = (V, E)$   $V$  (vertices) = จุด,  $E$  (edges) = เส้น  
adjacent อยู่ติดจาก เช่น  $(V, W)$   $W$  อยู่ติดจาก  $V$

กราฟไม่มีทิศทาง Undirect G

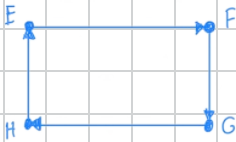


$F$  adjacent  $E$  เพราะมีคู่อันดับ  $(E, F)$   
แต่  $E$  ไม่ได้ adjacent กับ  $F$  เพราะไม่มีคู่อันดับ  $(F, E)$

Undirect Graph  $V = \{A, B, C, D\}$

$E = \{(A, B), (B, C), (C, D), (D, A)\}$  สลับกันได้

กราฟมีทิศทาง



$V = \{E, F, G, H\}$

$E = \{(E, F), (F, G), (G, H), (H, E)\}$  สลับกันไม่ได้

length = จำนวนของ edges

ถ้าไม่มี path edges = 0

path = เส้นทาง, เส้นเชื่อม

simple path คือ จาก vertex

แตกต่างกัน ไปจนถึงจุดเริ่มต้นกับจุดสิ้นสุด

จงเขียน path จาก vertex E ไป vertex H

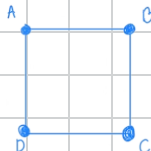
$E, F, G, H$  ถูก

อย่าลืม  $E \rightarrow F \rightarrow G \rightarrow H$  0 คือสั้น

$E$  ไป vertex E ที่ length ไม่น้อยกว่า 0

$E, F, G, H, E$

กราฟไม่มีทิศทาง Undirect G

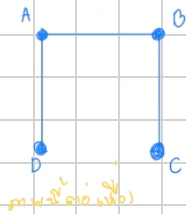


Connected path

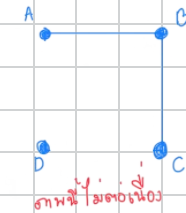
path  $\neq$  edges

|     |     |     |     |
|-----|-----|-----|-----|
| A B | B A | C A | D A |
| A C | B C | C B | D B |
| A D | B D | C D | D C |

สมมติต้องเขียน path ทุกคู่ของ vertex



|    |    |    |    |
|----|----|----|----|
| AB | BA | CA | DA |
| AC | BC | CB | DB |
| AD | BD | CD | DC |



|               |               |               |               |
|---------------|---------------|---------------|---------------|
| AB            | BA            | CA            | <del>DA</del> |
| AC            | BC            | CB            | <del>DB</del> |
| <del>AD</del> | <del>BD</del> | <del>CD</del> | <del>DC</del> |

$V = \{A\}$   
 $E = \{ \}$  ไม่ต่อเนื่อง  
 เป็นกราฟ

$V = \{ \}$   
 $E = \{ \}$  ไม่ต่อเนื่อง  
 เป็นกราฟ

cycle จุดเริ่มต้นกับจุดสิ้นสุดจุดเดียวกัน  
 acyclic ไม่มี cycles ไม่สามารถเดินทางกลับทาง  
 จุดเริ่มต้น  
 connected ถ้ามี path ระหว่างทุกคู่ของ  
 vertex

## Complete Graph



2 vertex

มี edge 1 เส้น



3 vertex

มี edge 3 เส้น



4 vertex

มี edge 6 เส้น



5 vertex

มี edge 10 เส้น

Complete Graph ที่มี 10 vertex จะมี edge ยาวน้อย ? = 45 เส้น

$$N * (N - 1) / 2 \quad (N = \text{vertex})$$

$$10 * (10 - 1) / 2 = 45$$

# Representation of Graphs

vertex อาจจะเขียน node

Adjacency Matrix ใช้ Array สดรวลิตี  
สูตร  $O(|V|^2)$

| u/v | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|-----|---|---|---|---|---|---|---|
| 1   | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 2   | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 3   | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 4   | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| 5   | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 6   | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7   | 0 | 0 | 0 | 0 | 0 | 1 | 0 |

Adjacency Matrix ไม่นิยมใช้ เพราะถ้ามันเปลี่ยนเงื่อนไขกำหนดความถี่

49 ข้อ มีข้อที่เป็น 1 อยู่ 12 ข้อ

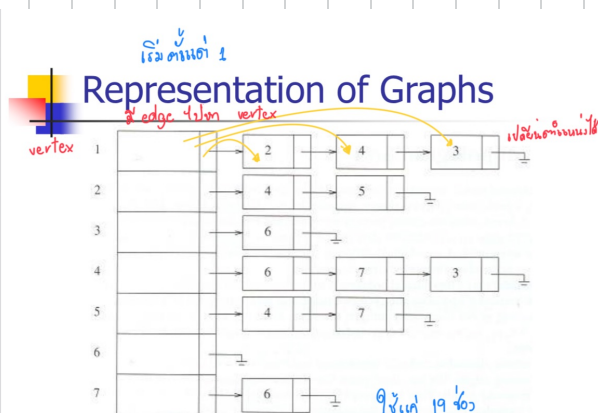
100 ข้อ มีข้อที่เป็น 1 อยู่  $\approx 12 \times 100 / 49 = 24.48 \%$

มีข้อที่เป็น 0 หรือไม่ได้ใช้ถามอยู่ 75.51 %

## Adjacency List ถักรวณไม่หาค่าแน่นอน

$O(|E| + |V|) = \text{เส้น} + \text{จุด}$

ex ถักรวณ Direct Graph มี 10 vertex มี edge 20 ถักรวณ Adjacency list = 30 ข้อ

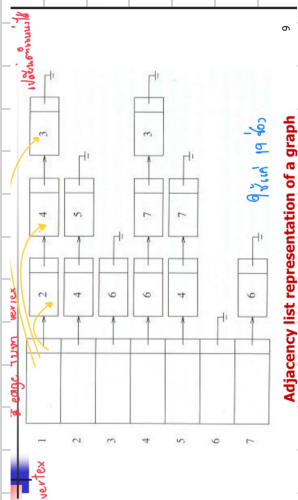


Adjacency list representation of a graph

# Topological Sort ทำได้ของ vertex

- \* ถ้ามี cycle เป็นไม่ได้ cycle = วนรอบ
- \* ต้องไม่มีทิศทางกลับจากจุดเริ่มต้น

1. แปลงกราฟให้เป็น Adjacency list now



2. Indegree Array อ: เรียงจาก จำนวน edge ที่เข้ามา vertex

| V | เริ่มต้น | row 1 | row 2 | row 3 | row 4 | row 5 |
|---|----------|-------|-------|-------|-------|-------|
| 1 | 0        | 0     | 0     | 0     | 0     | 0     |
| 2 | 1        | 0     | 0     | 0     | 0     | 0     |
| 3 | 2        | 1     | 1     | 1     | 0     | 0     |
| 4 | 3        | 2     | 1     | 1     | 0     | 0     |
| 5 | 1        | 1     | 0     | 0     | 0     | 0     |
| 6 | 3        | 1     | 1     | 1     | 2     | 0     |
| 7 | 2        | 2     | 2     | 1     | 0     | 0     |

$v = 1$

for each  $w$  adjacent to  $v$   $w = 2$

ไปต่อในข้อที่ 2

$w = 4$

$w = 3$

$v = 2$

for each  $w$  adjacent to  $v$   $w = 4$

$w = 5$

$v = 3$

for each  $w$  adjacent to  $v$   $w = 6$

$w = 7$

$v = 4$

for each  $w$  adjacent to  $v$   $w = 7$

$w = 7$

$w = 3$

ไปต่อที่ adjacent list ของ vertex  $v$

ลด indegree ของ vertex ที่มันมีค่าเป็น 0 ไปใส่ใน q

```

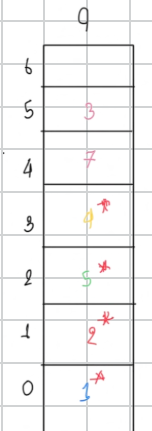
/* Pseudocode ขุดได้คิด ทำสิ่งที่ยากเพื่อ perform topological sort
void Graph::topsort() {
    Queue q(NUM_VERTICES);
    int counter = 0;
    Vertex v, w;

    q.makeEmpty();
    for each vertex v
        if (v.indegree == 0)
            q.enqueue(v);

    while (!q.isEmpty())
    {
        v = q.dequeue();
        v.topNum = ++counter; // assign next number

        for each w adjacent to v
            if (--w.indegree == 0)
                q.enqueue(w);
    }

    if (counter != NUM_VERTICES)
        throw CycleFound();
}
    
```



currentSize = 0  
Front = 0, Back = 0

# Shortest Path Algorithms

## Single Shortest Path Problem

เส้นทางที่สั้นที่สุด  
จากเริ่มต้นที่จุดเดียว

ไม่ใส่ค่าให้กับ edges ติดต่อกัน

## Shortest Path Algorithms

### Example :

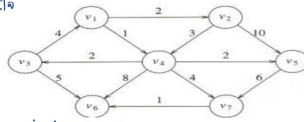
On the figure

- Cost of shortest weighted path from  $v_1$  to  $v_6$  is 6

- $v_1$  to  $v_4$  to  $v_7$  to  $v_6$

- Shortest unweighted path between these vertices is 2

- $v_1$  to  $v_4$  to  $v_6$



ไม่ใส่ค่าให้กับ edges ติดต่อกัน  
... นับเป็น 1 เดียว

## Unweighted Shortest Paths

ไม่มี cycle

ตัวไปต่อเลข 0 ไม่ใส่ edge ซ้ำ

$D_v$  (distance) = ระยะทาง 999

$P_v$  (path) = เส้นทาง 0

Shortest Path

1

|   |
|---|
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
| 6 |
| 7 |

|    |
|----|
| 2  |
| 10 |
| 1  |
| 9  |
| 7  |
| 6  |

|   |
|---|
| 4 |
| 5 |
| 6 |
| 6 |

5

7

2 Array

ระยะทาง เส้นทาง

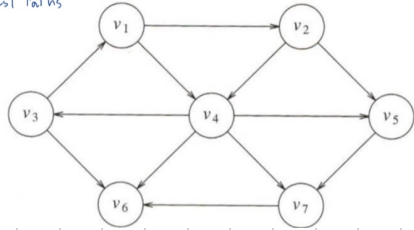
|     | know | $d_v$ | $p_v$ |
|-----|------|-------|-------|
| → 1 | T    | 1     | 3     |
| → 2 | T    | 2     | 1     |
| → 3 | T    | 0     | 0     |
| → 4 | F    | 2     | 1     |
| → 5 | F    | 3     | 2     |
| → 6 | T    | 1     | 3     |
| → 7 | F    | 3     | 4     |

|   |    |
|---|----|
| 6 | 7  |
| 5 | 5  |
| 4 | 4* |
| 3 | 2* |
| 2 | 6* |
| 1 | 1* |
| 0 | 3* |

## Unweighted Shortest Paths

- We are only interested in the number of edges contained on the path
- There are no weights on the edges

ไม่มี cycle



$V = 3$   
 $w = 1$   
 $w.dist = v.dist + 1$   
 $1 = 0 + 1$   
 $w.path = v$   
 $= 3$   
 $q.enqueue(w)$

$V = 1$   
 $w = 2$   
 $w.dist = v.dist + 1$   
 $2 = 1 + 1$   
 $w.path = v$   
 $= 1$   
 $q.enqueue(w)$

$V = 6$   
 $V = 2$   
 $w = 4$  if false  
 $w = 5$   
 $w.dist = v.dist + 1$   
 $3 = 2 + 1$   
 $w.path = v$   
 $= 2$   
 $q.enqueue(w)$

$w = 6$   
 $w.dist = v.dist + 1$   
 $1 = 0 + 1$   
 $w.path = v$   
 $= 3$   
 $q.enqueue(w)$

$w = 4$   
 $w.dist = v.dist + 1$   
 $2 = 1 + 1$   
 $w.path = v$   
 $= 1$   
 $q.enqueue(w)$

$V = 4$   
 $w = 3$   
 $w = 6$   
 $w = 5$   
 $w = 7$   
 $w.dist = v.dist + 1$   
 $3 = 2 + 1$   
 $w.path = v$   
 $= 4$   
 $q.enqueue(w)$

จงหาเส้นทางที่สั้นที่สุด จาก  $V_3$  ไป  $V_5$

ตอบ ระยะทางเท่ากับ 3

เส้นทาง  $V_3, V_1, V_2, V_5$

จงหาเส้นทางที่สั้นที่สุด จาก  $V_3$  ไป  $V_7$

ตอบ ระยะทางเท่ากับ 3

เส้นทาง  $V_3, V_1, V_4, V_7$

จงหาเส้นทางที่สั้นที่สุด จาก  $V_3$  ไป  $V_5$  ผ่าน  $V_4$  ไป  $V_5$

ตอบ ไม่สามารถหาเส้นทางได้



[illegible]

```

graph LR
    A[A] --> B[B]
    B[B] --> C[C]
    C[C] --> E[E]
    E[E] --> D[D]
    D[D] --> A[A]
    A[A] --> F[F]
    F[F] --> F[F]
    G[G] --> E[E]
    
```

|   | Known          | Dist (Dv) | Path (Pv)      |
|---|----------------|-----------|----------------|
| A | F              | 999 5     | <del>0</del> D |
| B | <del>0</del> T | 999 0     | 0              |
| C | <del>0</del> T | 999 1     | <del>0</del> B |
| D | <del>0</del> T | 999 2     | <del>0</del> C |
| E | <del>0</del> T | 999 1     | <del>0</del> B |
| F | F              | 999 2     | <del>0</del> E |
| G | <del>0</del> T | 999 1     | <del>0</del> D |

| Queue |
|-------|
| A     |
| F     |
| D     |
| G *   |
| E *   |
| C *   |
| B *   |

g. enquehe C

$$\begin{array}{r}
 1011 \\
 10 \overline{) 101113} \\
 \underline{10} \phantom{00} \\
 011 \phantom{00} \\
 \underline{10} \phantom{00} \\
 13 \phantom{00} \\
 \underline{10} \phantom{00} \\
 \underline{3}
 \end{array}$$

```

1 makeEmpty();
}

```

|    |  |   |
|----|--|---|
| 0  |  | E |
| 1  |  | E |
| 2  |  | E |
| 3  |  | E |
| 4  |  | E |
| 5  |  | E |
| 6  |  | E |
| 7  |  | E |
| 8  |  | E |
| 9  |  | E |
| 10 |  | E |